

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

*I am **M.Niharika(192365019)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:M.Niharika

Annexure A

EXPERIMENTS

Q1.You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

- (i) Accept an arithmetic expression with operators +, -, *, / and parentheses
- (ii) Use syntax-directed evaluation to compute values based on operator precedence
- (iii) Optimize evaluation by reducing unnecessary temporary computations
- (iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

$E \rightarrow E + T \quad \{ E.val = E1.val + T.val \}$

$E \rightarrow E - T \quad \{ E.val = E1.val - T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow T * F \quad \{ T.val = T1.val * F.val \}$

$T \rightarrow T / F \quad \{ T.val = T1.val / F.val \}$

$T \rightarrow F \quad \{ T.val = F.val \}$

$F \rightarrow (E) \quad \{ F.val = E.val \}$

$F \rightarrow digit \quad \{ F.val = digit \}$

Output:

```

1 #include <stdio.h>
2 #include <ctype.h>
3 char exp[100];
4 int pos = 0;
5 int t1();
6 int t2();
7 int f();
8 int t1() {
9     int value = t2();
10    while (exp[pos] == '+' || exp[pos] == '-') {
11        char op = exp[pos++];
12        int value2 = t1();
13        if (op == '+')
14            value += value2;
15        else
16            value -= value2;
17    }
18    return value;
19 }
20 int t2() {
21     int value = f();
22     while (exp[pos] == '*' || exp[pos] == '/') {
23         char op = exp[pos++];
24         int value2 = f();
25         if (op == '*')
26             value *= value2;
27         else
28             value /= value2;
29     }
30    return value;
31 }
32 int f() {
33     int value;
34     if (exp[pos] == '(') {
35         pos++;
36         value = t1();
37         pos++;
38     } else {
39         value = 0;
40         while (isdigit(exp[pos]))
41             value = value * 10 + (exp[pos++] - '0');
42     }
43    return value;
44 }
45 int main() {
46     printf("Enter arithmetic expression: ");
47     scanf("%s", exp);
48     printf("Result = %d\n", f());
49     return 0;
50 }

```

Enter arithmetic expression: (2+3)*4
Result = 20

Process exited after 19.99 seconds with return value 0
Press any key to continue . . .

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.

Write a C program that:

- (i) Evaluates a postfix expression
- (ii) Demonstrates S-attributed definition evaluation
- (iii) Shows how intermediate results are computed

Output:

```

1 #include <stdio.h>
2 #include <ctype.h>
3 int stack[100];
4 int top = -1;
5 void push(int x) {
6     stack[++top] = x;
7 }
8 int pop() {
9     return stack[top--];
10 }
11 int main() {
12     char exp[100];
13     int i, a, b, result;
14     printf("Enter postfix expression: ");
15     scanf("%s", exp);
16     for (i = 0; exp[i] != '\0'; i++) {
17         if (isdigit(exp[i])) {
18             push(exp[i] - '0');
19         } else {
20             b = pop();
21             a = pop();
22             switch (exp[i]) {
23                 case '*': result = a * b; break;
24                 case '-': result = a - b; break;
25                 case '+': result = a + b; break;
26                 case '/': result = a / b; break;
27             }
28             push(result);
29         }
30     }
31     printf("Result = %d\n", pop());
32     return 0;
33 }

```

Enter postfix expression: 234**
Result = 14

Process exited after 31.71 seconds with return value 0
Press any key to continue . . .

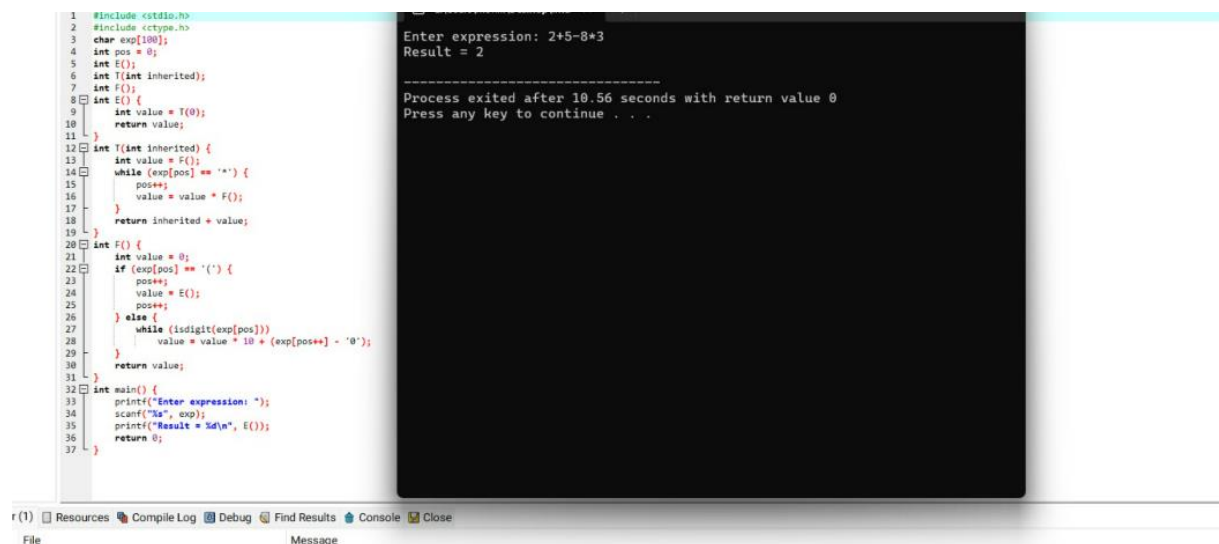
Q3.You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

- (i).Evaluates a simple expression like $a + b * c$
- (ii) Uses function parameters to simulate inherited attributes
- (iii). Ensures correct precedence handling
- (iv). Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

Output:



The image shows a screenshot of a C program and its execution output. The C program is a top-down parser for the expression $2+5-8*3$. It uses L-attributed definitions with inherited attributes. The program defines two functions: `E()` for evaluating expressions and `F()` for evaluating factors. The `E()` function handles addition and subtraction, while the `F()` function handles multiplication and division. The program uses a global variable `pos` to track the current position in the input string. The output shows the program evaluating the expression $2+5-8*3$ and returning the result `2`. The program also displays the message "Process exited after 10.56 seconds with return value 0" and "Press any key to continue . . .".

```
1 #include <stdio.h>
2 #include <ctype.h>
3 char exp[100];
4 int pos = 0;
5 int E();
6 int T(int inherited);
7 int F();
8 int E() {
9     int value = T(0);
10    return value;
11 }
12 int T(int inherited) {
13     int value = F();
14     while (exp[pos] == '*' || exp[pos] == '/') {
15         pos++;
16         value = value * F();
17     }
18     return inherited + value;
19 }
20 int F() {
21     int value = 0;
22     if (exp[pos] == '(') {
23         pos++;
24         value = E();
25         pos++;
26     } else {
27         while (isdigit(exp[pos])) {
28             value = value * 10 + (exp[pos++] - '0');
29         }
30     }
31     return value;
32 }
33 int main() {
34     printf("Enter expression: ");
35     scanf("%s", exp);
36     printf("Result = %d\n", E());
37     return 0;
38 }
```

Enter expression: 2+5-8*3
Result = 2

Process exited after 10.56 seconds with return value 0
Press any key to continue . . .

Q4.In a compiler, it is important to check whether two type expressions are equivalent.

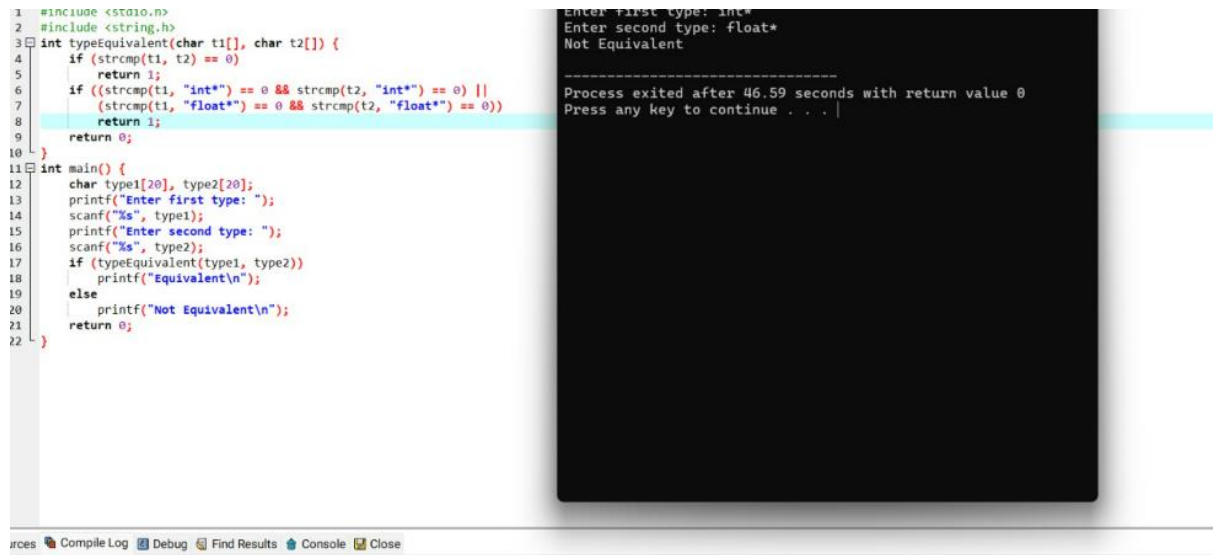
Write a C program that:

- (i).Accepts two type expressions (e.g., int, float, int*)
- (ii).Compares them for **type equivalence**
- (iii).Displays whether they are:
 - (a).Equivalent

(b).Not equivalent

Extend the logic to handle pointer types (`int*`, `float*`).

Output:



The image shows a C program in a code editor and its execution output in a terminal window. The code defines a function `typeEquivalent` that checks if two type strings are equivalent. It handles basic types (`int`, `float`) and pointer types (`int*`, `float*`). The `main` function prompts the user to enter two types and prints the result.

```
1 #include <stdio.h>
2 #include <string.h>
3 int typeEquivalent(char t1[], char t2[]) {
4     if (strcmp(t1, t2) == 0)
5         return 1;
6     if ((strcmp(t1, "int*") == 0 && strcmp(t2, "int*") == 0) ||
7         (strcmp(t1, "float*") == 0 && strcmp(t2, "float*") == 0))
8         return 1;
9     return 0;
10 }
11 int main() {
12     char type1[20], type2[20];
13     printf("Enter first type: ");
14     scanf("%s", type1);
15     printf("Enter second type: ");
16     scanf("%s", type2);
17     if (typeEquivalent(type1, type2))
18         printf("Equivalent\n");
19     else
20         printf("Not Equivalent\n");
21     return 0;
22 }
```

The terminal output shows the program being run with `int*` and `float*` as inputs, resulting in "Not Equivalent". It also shows the process exiting after 46.59 seconds.

```
Enter first type: int*
Enter second type: float*
Not Equivalent

-----
Process exited after 46.59 seconds with return value 0
Press any key to continue . . .
```

Q5.You are building a semantic analyzer that detects type errors in arithmetic expressions.

Write a C program that:

- (i). Accepts two operands and their types
- (ii). Checks whether the operation (e.g., +, *) is valid
- (iii). Reports:
 - (a).Valid expression
 - (b).Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

Output:

```

1 #include <stdio.h>
2 #include <string.h>
3 int validOperation(char type1[], char op, char type2[]) {
4     if (strcmp(type1, "int") == 0 && strcmp(type2, "int") == 0)
5         return 1;
6     if (strcmp(type1, "float") == 0 && strcmp(type2, "float") == 0)
7         return 1;
8     return 0;
9 }
10 int main() {
11     char type1[20], type2[20], op;
12     printf("Enter first operand type: ");
13     scanf("%s", type1);
14     printf("Enter operator (+, *, -, /): ");
15     scanf("%c", &op);
16     printf("Enter second operand type: ");
17     scanf("%s", type2);
18     if (validOperation(type1, op, type2))
19         printf("Valid expression\n");
20     else
21         printf("Type Error: Incompatible operands\n");
22     return 0;
23 }

```

```

C:\Users\monik\Desktop >
Enter first operand type: int
Enter operator (+, *, -, /): *
Enter second operand type: int
Valid expression

-----
Process exited after 26.01 seconds with return value 0
Press any key to continue . . .

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

